

# XY Cargo Zambia Project Documentation

Date: 2026-05-28

## 1. Project Overview

XY Cargo Zambia is a React and Supabase freight operations platform. It combines a public marketing website with authenticated portals for customers, agents, warehouse staff, drivers, finance, compliance, marketing, support, and administrators.

The application supports the full shipment lifecycle:

- ? Public visitors can view services, pricing, tracking, support, and company information.
- ? Customers can place orders, request deliveries, track parcels, manage payments, raise claims, and contact support.
- ? Agents can manage customers, shipments, payments, commissions, sourcing, and supplier requests.
- ? Warehouse users can receive parcels, manage inspections, consolidate shipments, update parcel status, and coordinate deliveries.
- ? Drivers can view deliveries, report incidents, and track performance.
- ? Finance users can manage payments, invoices, accounts receivable, wallets, commissions, COD settlements, billing, reconciliation, and reports.
- ? Compliance users can review queues and record compliance charges.
- ? Support users can manage tickets, escalations, customer profiles, SLAs, claims, knowledge base records, bulk SMS, and problem parcels.
- ? Marketing users can manage campaigns, leads, analytics, content, promotions, social activity, email, budgets, sales, and reports.
- ? Admin users can manage users, roles, branches, customers, drivers, receivers, missions, transactions, settings, and reports.

## 2. Technology Stack

- ? Vite for development and production builds.
- ? React 18 with TypeScript.
- ? React Router for client-side routing.
- ? TanStack Query for async data fetching and caching.
- ? Supabase for authentication, database, generated database types, storage, and edge functions.
- ? Tailwind CSS for styling.
- ? shadcn/ui style components built on Radix UI primitives.
- ? Lucide React for icons.
- ? Recharts for charts.
- ? React Hook Form and Zod for forms and validation.
- ? jsPDF and jsPDF AutoTable for PDF generation.
- ? XLSX for spreadsheet import or export workflows.
- ? Vitest and Testing Library for tests.

## 3. Repository Structure

```

.
|-- src/
|   |-- assets/           Static frontend assets imported by React
|   |-- components/       Shared UI, layout, auth, public, and domain components
|   |-- content/          CMS defaults and public content defaults
|   |-- hooks/            Shared React hooks
|   |-- integrations/supabase/ Supabase client and generated database types
|   |-- lib/              Business logic helpers and portal data utilities
|   |-- pages/            Route-level pages grouped by portal/domain
|   |-- test/             Test helpers and test setup
|-- supabase/
|   |-- functions/        Supabase Edge Functions
|   |-- migrations/       Database schema and data migrations
|   |-- manual/           Manual database scripts
|   |-- manual_scripts/   Additional manual database utilities
|-- public/              Static public files served by Vite
|-- docs/                Project documentation
|-- dist/                Production build output
|-- package.json          Scripts and dependencies
|-- vite.config.ts        Vite configuration
|-- tailwind.config.ts    Tailwind configuration
`-- README.md             Quick start summary

```

## 4. Runtime Architecture

The frontend is a single-page application. `src/App.tsx` owns the top-level providers, routing tree, lazy-loaded pages, public layout, authenticated dashboard layout, role guards, and redirects.

Core runtime layers:

- ? Public site layer: ``PublicLayout`` wraps public marketing and service pages.
- ? Auth layer: login, register, reset password, and shared auth context.
- ? Protected app layer: ``DashboardLayout`` wraps authenticated operational routes.
- ? Role guard layer: ``RequireRole`` and ``RequireNonCustomer`` restrict portal access.
- ? Data layer: Supabase client, generated database types, domain helpers in ``src/lib``, and React Query usage inside pages and components.
- ? UI layer: shared components in ``src/components``, Tailwind utilities, Radix/shadcn primitives, and Lucide icons.

## 5. Environment and Configuration

The Supabase client is configured in:

```
src/integrations/supabase/client.ts
```

The app reads:

- ? ``VITE_SUPABASE_URL``
- ? ``VITE_SUPABASE_PUBLISHABLE_KEY``

The current client also includes fallback Supabase project values so the app can run when those environment variables are not set. Sensitive service role keys and third-party API secrets should not be committed into frontend code. Runtime secrets belong in Supabase Edge Function secrets or secured database settings.

Useful commands:

```
npm install
npm run dev
npm run build
npm run preview
npm run lint
npm run test
```

## 6. Authentication and Roles

Authentication is Supabase-based. The shared auth context is defined in:

```
src/components/auth/AuthContext.tsx
```

The app uses role-aware routing. Major role groups include:

- ? Customer
- ? Agent
- ? Driver
- ? Warehouse
- ? Finance
- ? Compliance
- ? Marketing
- ? Support
- ? Admin or non-customer staff

Route protection is handled in the routing tree and shared auth components. Customer-only routes are separated from internal staff routes. Non-customer users are routed through the dashboard layout and role-specific sections.

## 7. Public Website

The public website is the customer-facing entry point for XY Cargo Zambia. It includes:

- ? Home page.
- ? Services overview.
- ? Product sourcing service.
- ? Supplier payment facilitation.
- ? Customs clearance.
- ? Export service.
- ? How it works.
- ? Gallery.
- ? Podcast.
- ? Blog.
- ? Privacy policy.
- ? Terms.
- ? Refund policy.
- ? FAQ.
- ? Public calculator.
- ? Pricing.

- ? Locations.
- ? Public tracking.
- ? Support.
- ? About.
- ? Join us.
- ? Shop.
- ? Language page.

Public content defaults live in:

```
src/content/cmsDefaults.ts  
src/lib/cmsContent.ts
```

The CMS editor route allows internal users to manage editable public content.

## 8. Customer Portal

Customer routes cover day-to-day customer self-service:

- ? Dashboard and overview.
- ? Inbox.
- ? Shipments.
- ? Place order.
- ? Request delivery.
- ? Tracking.
- ? Problem parcels.
- ? Claims.
- ? Refunds.
- ? Warehouse address.
- ? Saved addresses.
- ? Pay on behalf.
- ? Payments.
- ? Profile and security.
- ? Support tickets and ticket detail.
- ? Reports.
- ? Sourcing.
- ? Supplier requests.
- ? Custom payment.

Key route files are under:

```
src/pages/customer/
```

## 9. Agent Portal

Agents can create and manage customer work on behalf of clients. Agent features include:

- ? Agent dashboard.

- ? Shipments.
- ? Request delivery.
- ? Customer list and customer creation.
- ? Place order.
- ? Payments.
- ? Commissions.
- ? Pricing.
- ? Tracking.
- ? Refunds.
- ? Reports.
- ? Sourcing.
- ? Support and ticket detail.
- ? Settings.
- ? Withdrawals.
- ? Supplier requests.
- ? Custom payment.

Admin-facing agent management also includes agent list, commissions, and reports.

Key files:

```
src/pages/agent/  
src/pages/agents/  
src/lib/agentPortal.ts
```

## 10. Warehouse Portal

Warehouse operations manage parcel intake, movement, consolidation, status updates, and delivery handoff.

Warehouse features include:

- ? Warehouse dashboard.
- ? All parcels.
- ? All shipments.
- ? Pending shipments.
- ? Receive parcel.
- ? Status updates.
- ? Problem parcels.
- ? Consolidation.
- ? Inspection uploads.
- ? Outgoing containers.
- ? Warehouse customers.
- ? Warehouse staff.
- ? Warehouse drivers.
- ? Deliveries.
- ? Delivery requests.
- ? Create shipment.

Key files:

```
src/pages/warehouse/  
src/lib/warehouseTabFilters.ts  
src/lib/parcelWorkflow.ts  
src/lib/shipmentNotes.ts
```

## 11. Driver Portal

Driver routes focus on delivery execution and field reporting:

- ? Driver dashboard.
- ? Delivery updates.
- ? Performance.
- ? Incidents.
- ? Settings.

Key files:

```
src/pages/driver/  
src/pages/drivers/  
src/lib/driverPortal.ts
```

## 12. Finance Portal

Finance manages money movement, reporting, statements, and settlement workflows.

Finance features include:

- ? Finance dashboard.
- ? Payments.
- ? Claims.
- ? Invoices.
- ? COD settlements.
- ? Commissions.
- ? Reports.
- ? Payment history.
- ? Accounts receivable.
- ? Reconciliation.
- ? Billing.
- ? Client statements.
- ? Wallets.
- ? Settings.

Important finance files:

```
src/pages/finance/  
src/lib/financePortal.ts  
src/lib/currencyDisplay.ts  
supabase/migrations/20260515113000_fix_compliance_currency_finance_precision.sql  
supabase/migrations/20260528124000_fix_default_currency_expense_amount_drift.sql
```

Finance precision handling:

- ? Compliance charge finance sync preserves original amount and original currency.
- ? Default-currency expenses should not be converted against their own exchange rate.
- ? Dashboard and report displays use shared currency display helpers to remove small historical decimal drift from whole-number amounts.
- ? See `docs/FINANCE\_PRECISION\_FIX.md` for the detailed fix notes.

## 13. Compliance Portal

Compliance users handle shipment compliance review and charge recording.

Compliance features include:

- ? Compliance dashboard.
- ? Compliance queue.
- ? Compliance charges.

Key files:

```
src/pages/compliance/  
supabase/migrations/20260515113000_fix_compliance_currency_finance_precision.sql
```

Compliance charge records can sync into finance expenses. The sync function stores:

- ? Base finance amount.
- ? Original amount.
- ? Original currency.
- ? Expense type.
- ? Expense date.
- ? Description.

## 14. Support Portal

Support users manage customer issues, internal notes, escalations, and operational support.

Support features include:

- ? Dashboard.
- ? Tickets.
- ? Ticket detail.
- ? Department queues.
- ? My tickets.
- ? Create ticket.
- ? Escalated tickets.
- ? Customer profiles.
- ? SLA monitoring.
- ? Reports.
- ? Problem parcels.
- ? Sourcing requests.

- ? Claims.
- ? Internal notes.
- ? Knowledge base.
- ? Bulk SMS.
- ? Supplier requests.

Key files:

```
src/pages/support/
src/lib/supportTickets.ts
```

## 15. Marketing Portal

Marketing covers campaigns, analytics, leads, sales, social media, and reporting.

Marketing features include:

- ? Dashboard.
- ? Campaigns.
- ? Leads.
- ? Content.
- ? Analytics.
- ? Promotions.
- ? Social.
- ? Email.
- ? Budget and ROI.
- ? Sales.
- ? Reports.
- ? Settings.

Key files:

```
src/pages/marketing/
src/lib/marketingMetrics.ts
supabase/migrations/20260515102000_fix_marketing_accuracy_and_cost_entries.sql
```

Marketing analytics excludes preview-builder traffic. Campaign records support manual cost entries and Meta-synced campaign rows.

## 16. Admin and Settings

Administrative routes cover core management areas:

- ? Dashboard.
- ? Profile.
- ? CMS content settings.
- ? Customers.
- ? Drivers.
- ? Receivers.



- ? Missions.
- ? Transactions.
- ? Users.
- ? Roles.
- ? Staff portal assignments.
- ? Reports.
- ? Branches.
- ? Warehouses.
- ? General system settings.

Settings routes include:

- ? Shipping rates.
- ? Shipping.
- ? Payments.
- ? General.
- ? Covered places.
- ? Areas.
- ? Delivery time.
- ? Packages.
- ? Product types.
- ? Pickup destinations.
- ? Currencies.
- ? Localization.
- ? Notifications.
- ? Google settings.
- ? Backup.
- ? Themes.
- ? Theme settings.
- ? API secrets.
- ? Currency management.

Key files:

```
src/pages/settings/  
src/pages/users/  
src/pages/roles/  
src/pages/customers/  
src/pages/drivers/  
src/pages/receivers/  
src/pages/missions/  
src/pages/transactions/  
src/pages/reports/
```

## 17. Supabase Backend

The Supabase folder contains database migrations, manual scripts, and edge functions.

Important folders:

supabase/migrations/  
supabase/functions/  
supabase/manual/  
supabase/manual\_scripts/

Migrations define and adjust database tables, functions, policies, indexes, sync logic, and data cleanup. Before deploying frontend changes that depend on database changes, apply the relevant migrations to the target Supabase project.

Generated database types are stored in:

src/integrations/supabase/types.ts

If schema changes are made, regenerate or update these types so TypeScript stays aligned with the database.

## 18. Supabase Edge Functions

Current edge functions:

- ? `admin-create-user`: Staff/admin user creation workflow.
- ? `admin-delete-user`: Staff/admin user deletion workflow.
- ? `lipila-callback`: Lipila payment callback handler.
- ? `lipila-payment`: Lipila payment initiation.
- ? `marketing-automation-trigger`: Marketing automation trigger.
- ? `public-tracking-lookup`: Public tracking lookup endpoint.
- ? `publish-social-post`: Social post publishing.
- ? `send-email`: Email delivery.
- ? `send-notification`: App notification delivery.
- ? `send-sms`: SMS delivery.
- ? `shipsgo-tracking`: ShipsGo tracking integration.
- ? `sms-diagnostic`: SMS diagnostics.
- ? `sync-social-metrics`: Social metrics sync.
- ? `update-exchange-rates`: Currency exchange rate updates.
- ? `verify-recaptcha`: reCAPTCHA verification.
- ? `wallet-payment`: Wallet payment handling.

Edge functions live under:

supabase/functions/

## 19. External Integrations

The project integrates with several external systems:

- ? Supabase Auth and database.
- ? Lipila for payment processing.
- ? ShipsGo for tracking.
- ? SMS provider through Supabase functions.
- ? Email provider through Supabase functions.

- ? Social publishing and metrics sync.
- ? reCAPTCHA verification.
- ? Exchange-rate update workflow.
- ? Google and analytics settings.

Integration secrets should be configured outside frontend source code. Use Supabase Edge Function secrets or secured settings pages where the app already provides that pattern.

## 20. Currency and Financial Data Rules

The system supports multiple currencies and keeps a default system currency. Finance records may store:

- ? ``amount``: base/default currency amount used in finance totals.
- ? ``original_amount``: original entered amount.
- ? ``original_currency``: original entered currency.

Rules:

- ? If the entered currency is the default currency, store and display the exact entered amount.
- ? If the source currency rate equals the default rate, do not convert.
- ? If conversion is needed, use configured active currency rates.
- ? Preserve original amount and currency for auditability.
- ? Use shared display helpers for totals that may have legacy minor drift.

Relevant files:

```
src/lib/currencyDisplay.ts
src/hooks/useCurrencyContext.tsx
src/pages/Dashboard.tsx
src/pages/finance/FinanceDashboard.tsx
src/pages/finance/FinanceReports.tsx
```

## 21. Build and Deployment

Development:

```
npm install
npm run dev
```

Production build:

```
npm run build
```

Preview built output:

```
npm run preview
```

Deployment checklist:

- ? Install dependencies with the lockfile in the target environment.
- ? Apply required Supabase migrations.
- ? Confirm required Supabase environment variables and secrets.

- ? Run lint and tests where practical.
- ? Run `npm run build`.
- ? Deploy the generated `dist/` output.
- ? Verify public routes, login, role-based dashboards, payment flows, tracking, and key finance totals.

## 22. Testing and Quality Checks

Available commands:

```
npm run lint
npm run test
npm run build
```

Recommended manual smoke checks:

- ? Public home page loads.
- ? Public tracking page loads.
- ? Login works.
- ? Customer dashboard loads.
- ? Admin dashboard loads.
- ? Finance dashboard totals render correctly.
- ? Warehouse receive and status pages load.
- ? Support ticket pages load.
- ? Marketing analytics excludes preview-builder traffic.
- ? Mobile view has no removed PWA/mobile-bottom-navigation artifacts.

## 23. PWA Status

PWA features have been removed from the project. The app should behave as a standard responsive web application.

Removed or deprecated PWA areas include:

- ? Web app manifests.
- ? Service worker.
- ? Push notification prompt.
- ? PWA manager utilities.
- ? PWA icon and splash artifacts.
- ? Mobile sticky bottom navigation component.

When adding new public assets, avoid reintroducing service worker registration, push prompts, manifest links, or install banners unless the PWA feature is intentionally restored.

## 24. Documentation Files

Existing documentation:

- ? `docs/AUTOMATIC\_PDF\_DOWNLOAD.md`
- ? `docs/FINANCE\_PRECISION\_FIX.md`
- ? `docs/INVOICE\_LOGO\_IMPLEMENTATION.md`

? `docs/PROJECT\_DOCUMENTATION.md`

Generated PDF documentation:

? `docs/FINANCE\_PRECISION\_FIX.pdf`

? `docs/PROJECT\_DOCUMENTATION.pdf`

## 25. Maintenance Notes

? Keep route documentation aligned with `src/App.tsx`.

? Keep backend documentation aligned with `supabase/functions/` and `supabase/migrations/`.

? Regenerate Supabase TypeScript types after schema changes.

? Update finance documentation whenever currency conversion, display formatting, or expense sync logic changes.

? Update deployment notes if the hosting provider, environment variable strategy, or build output changes.

? Avoid editing unrelated pending files when making targeted fixes.

## 26. Troubleshooting

If the app fails to start:

? Run `npm install`.

? Confirm Node.js and npm are installed.

? Confirm Vite can read the project root.

? Check TypeScript errors in the terminal.

If Supabase data does not load:

? Confirm Supabase URL and publishable key.

? Confirm the target database has the latest migrations.

? Check row level security policies.

? Check browser console network errors.

If role routes redirect unexpectedly:

? Confirm the logged-in user's role.

? Confirm role assignment records and portal assignment records.

? Check route guards in `src/App.tsx`.

If finance totals look wrong:

? Check the default currency configuration.

? Check active currency exchange rates.

? Check `finance\_expenses.amount`, `original\_amount`, and `original\_currency`.

? Confirm the latest finance precision migrations have been applied.

? Confirm frontend display helpers are used by the affected dashboard or report.

If edge functions fail:

? Check Supabase function logs.

? Confirm function secrets are configured.

- ? Confirm request payloads match function expectations.
- ? Confirm CORS handling where browser calls are involved.